

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG

1.1. Mục tiêu và Môi trường phát triển

Trong khuôn khổ môn học Hệ quản trị cơ sở dữ liệu, nhóm chúng em đã quyết định thực hiện đề tài xây dựng hệ thống quản lý đăng ký học phần của sinh viên theo hình thức tín chỉ. Mục tiêu cốt lõi của đề án này là tạo ra một cơ sở dữ liệu hoàn chỉnh, có khả năng vận hành sát với thực tế, giúp quản lý thông tin sinh viên, giáo viên, lịch học và giải quyết trọn vẹn bài toán đăng ký môn học vốn rất quen thuộc trong trường đại học. Để phát triển đề án này, nhóm lựa chọn sử dụng hệ quản trị cơ sở dữ liệu MySQL phiên bản 8.0 trở lên cùng với phần mềm thao tác trực quan là MySQL Workbench. Về mặt cấu hình lưu trữ, cơ sở dữ liệu được nhóm thiết lập sử dụng bộ máy lưu trữ InnoDB. Nhóm quyết định chọn bộ máy lưu trữ này vì nó hỗ trợ rất tốt cho tính năng xử lý giao dịch và khả năng khóa từng dòng dữ liệu, giúp giải quyết an toàn các tình huống có nhiều sinh viên cùng lúc bấm nút đăng ký. Đi kèm với đó, mức độ cô lập dữ liệu mặc định được nhóm giữ nguyên ở chuẩn REPEATABLE READ nhằm đảm bảo tính chính xác tuyệt đối, ngăn chặn tình trạng sai lệch dữ liệu khi chạy các báo cáo thống kê. Dữ liệu lưu trữ trong hệ thống được định dạng bằng bộ mã ký tự utf8mb4 nhằm hỗ trợ thao tác nhập liệu và hiển thị ngôn ngữ tiếng Việt có dấu một cách chuẩn xác nhất. Cuối cùng, để đảm bảo an toàn thông tin, toàn bộ mật khẩu đăng nhập của tài khoản sinh viên và giáo viên đều không được lưu dưới dạng văn bản thường mà được hệ thống tự động mã hóa hoàn toàn theo chuẩn SHA2-256 ngay từ lúc tạo mới.

1.2. Các ràng buộc dữ liệu quan trọng

Để đảm bảo hệ thống luôn vận hành trơn tru và không bị chứa các thông tin rác hay dữ liệu sai lệch do người dùng vô tình nhập vào, nhóm đã phân tích và thiết lập một bộ quy tắc ràng buộc dữ liệu cực kỳ chặt chẽ ngay từ khâu thiết kế bảng. Thứ nhất là quy định về quản lý tín chỉ. Hệ thống được lập trình để giới hạn số lượng tín chỉ của mỗi môn học chỉ được phép dao động trong khoảng hợp lệ từ một đến mười tín chỉ. Cùng với đó, để tránh việc sinh viên đăng ký quá tải, nhóm đã bổ sung thêm một điều kiện kiểm tra giới hạn ngay trong thủ tục đăng ký, đảm bảo mỗi sinh viên chỉ được phép đăng ký tối đa hai mươi lăm tín chỉ trong một học kỳ. Nếu sinh viên chọn số môn vượt quá mức quy định này thì hệ thống sẽ lập tức báo lỗi và từ chối lưu dữ liệu vào bảng đăng ký.

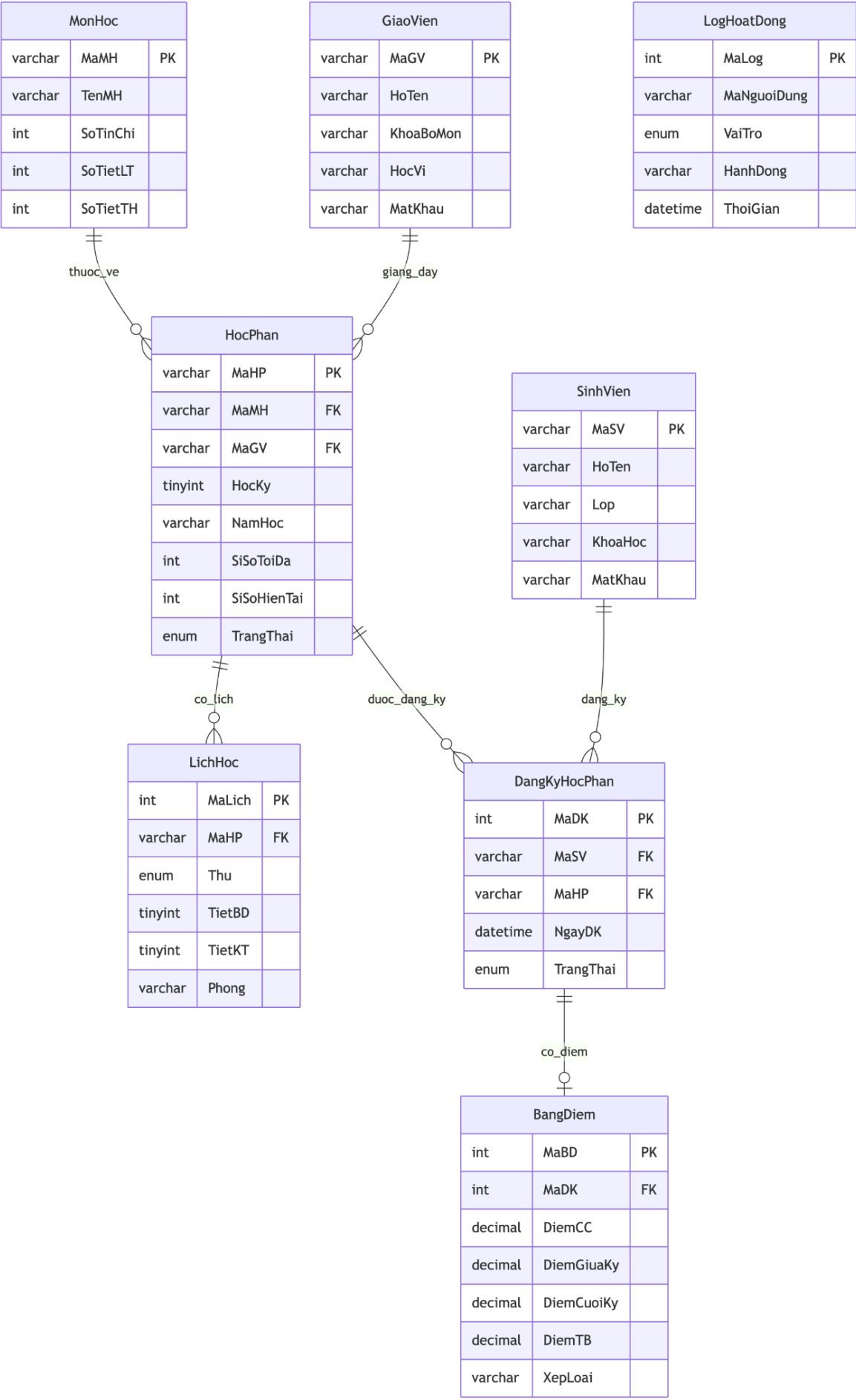
Thứ hai là các ràng buộc liên quan đến quản lý lớp học và lịch học. Nhóm cài đặt quy định rất rõ ràng rằng biến số đếm số lượng sinh viên hiện tại trong một lớp học không bao giờ được phép lớn hơn con số sĩ số tối đa mà nhà trường đã đặt ra ban đầu. Ngoài ra, trước khi lưu thông tin đăng ký vào cơ sở dữ liệu, hệ thống sẽ gọi một

hàm tự động để đối chiếu lịch học xem có bị trùng thời gian với các môn đã được sinh viên đăng ký trước đó hay không, nếu phát hiện trùng lịch thì giao dịch cũng sẽ bị hệ thống tự động hủy bỏ. Để ngăn chặn triệt để tình trạng một người đăng ký hai lần cho cùng một môn học hoặc gây ra lỗi ghi đè dữ liệu, nhóm đã thiết lập một khóa duy nhất ghép trên cả hai cột thông tin là mã sinh viên và mã học phần. Nhờ thiết lập này, hệ thống luôn hiểu rằng tại mỗi một lớp, mỗi sinh viên chỉ có duy nhất một lượt ghi nhận thành công.

Cuối cùng là những ràng buộc kiểm soát về mặt điểm số. Tất cả các cột điểm thành phần của sinh viên đạt được đều bắt buộc phải được giáo viên nhập trong thang điểm chuẩn từ mức không cho đến mức mười. Đặc biệt, nhằm hỗ trợ giáo viên thao tác nhanh chóng và giảm thiểu tối đa các sai sót do việc tính nhầm thủ công, nhóm đã ứng dụng kỹ thuật tạo cột tự động sinh để quản lý dữ liệu cho cột điểm trung bình. Cột điểm trung bình này được hệ thống gắn sẵn một công thức tính toán cố định theo đúng quy chế: lấy mười phần trăm điểm chuyên cần, cộng với ba mươi phần trăm điểm giữa kỳ và sáu mươi phần trăm điểm cuối kỳ. Nhờ tính năng này, điểm tổng kết môn học của sinh viên sẽ luôn tự động được tính toán lại và xuất ra con số chính xác nhất ngay tại thời điểm giáo viên vừa nhập hoặc sửa xong các đầu điểm thành phần.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1. Sơ đồ thực thể liên kết ERD và Quan hệ bảng



2.1. Sơ đồ thực thể liên kết ERD và Quan hệ bảng

Dựa trên những yêu cầu thực tế của việc quản lý đào tạo theo hình thức tín chỉ tại trường đại học, nhóm chúng em đã tiến hành phân tích nghiệp vụ và xây dựng sơ đồ thực thể liên kết ERD để hình dung tổng quan toàn bộ cấu trúc dữ liệu của hệ thống. Bước đầu tiên trong quá trình thiết kế, nhóm đã xác định các đối tượng cốt lõi tham gia vào quy trình quản lý bao gồm thực thể sinh viên, thực thể giáo viên và thực thể môn học. Tuy nhiên trong môi trường đào tạo thực tế, một môn học nền tảng có thể được nhà trường mở thành rất nhiều lớp khác nhau trong từng học kỳ tùy thuộc vào số lượng nhu cầu đăng ký. Do đó, nhóm quyết định tách thêm một thực thể chi tiết hơn mang tên là học phần để đại diện cho từng lớp học cụ thể đang được giảng dạy tại trường.

Trong toàn bộ sơ đồ thiết kế này, mối quan hệ phức tạp nhất chính là sự tương tác qua lại giữa sinh viên và học phần. Theo như phân tích nghiệp vụ, một sinh viên có nhu cầu đăng ký rất nhiều học phần khác nhau trong một học kỳ, và ngược lại, một học phần cũng sẽ tiếp nhận nhiều sinh viên vào học cùng một lúc. Để giải quyết triệt để mối quan hệ nhiều nhiều này theo đúng nguyên tắc chuẩn hóa cơ sở dữ liệu, nhóm đã thiết kế thêm một thực thể trung gian mang tên là đăng ký học phần. Thực thể trung gian này đóng vai trò giống như một trạm luân chuyển dữ liệu, giúp phân rã mối quan hệ phức tạp ban đầu thành các mối quan hệ một nhiều đơn giản hơn, từ đó hệ thống có thể dễ dàng quản lý và truy vết được chính xác sinh viên nào đang ghi danh vào những lớp học nào.

Bên cạnh các thực thể chính yếu, nhóm cũng chủ động bổ sung thêm các thực thể phụ trợ nhằm hoàn thiện toàn bộ hệ sinh thái quản lý dữ liệu. Các thực thể này bao gồm lịch học để quản lý thời khóa biểu chi tiết của từng lớp, bảng điểm để lưu trữ toàn bộ điểm số thành phần, và một thực thể nhật ký hoạt động đóng vai trò theo dõi lịch sử thao tác của người dùng. Từ sơ đồ mức khái niệm ban đầu này, nhóm chúng em đã tiến hành chuyển đổi trực tiếp thành sơ đồ quan hệ bảng chi tiết. Tại bước này, nhóm xác định rõ từng trường dữ liệu cho mỗi bảng, thiết lập các khóa chính Primary Key để định danh độc nhất cho mỗi dòng dữ liệu và vẽ các đường liên kết bằng khóa ngoại Foreign Key nối chặt chẽ giữa các bảng với nhau. Việc xây dựng một sơ đồ quan hệ bảng chuẩn xác ngay từ khâu thiết kế ban đầu đã tạo ra một nền tảng vô cùng vững chắc để nhóm có thể tự tin bước vào giai đoạn viết mã lệnh khởi tạo cơ sở dữ liệu vật lý trên phần mềm.

2.2. Cấu trúc các bảng dữ liệu

Để hiện thực hóa sơ đồ thiết kế thực thể liên kết thành một cơ sở dữ liệu vật lý hoàn chỉnh có thể chạy được trên phần mềm, nhóm chúng em đã tiến hành viết mã lệnh tạo lập cấu trúc Database bao gồm tám bảng dữ liệu chính thức. Nhóm phân chia các bảng này thành từng cụm chức năng cụ thể để dễ dàng quản lý. Cụm đầu tiên là các bảng danh mục cơ bản bao gồm bảng SinhVien, bảng GiaoVien và bảng MonHoc. Bảng SinhVien và bảng GiaoVien có nhiệm vụ lưu trữ thông tin cá nhân và tài khoản đăng nhập của người dùng vào hệ thống, trong đó mật khẩu đã được nhóm thiết lập chuẩn mã hóa an toàn. Bảng MonHoc dùng để chứa danh sách các môn học nền tảng của trường. Kế tiếp là bảng HocPhan, đây là bảng lưu trữ thông tin chi tiết về các lớp học cụ thể được mở ra từ các môn học nền tảng trong từng học kỳ. Tại bảng HocPhan này, nhóm có thiết lập các điều kiện kiểm tra nghiêm ngặt để đảm bảo số lượng sinh viên hiện tại trong lớp không bao giờ được phép vượt quá mức sĩ số tối đa quy định. Đi kèm với học phần là bảng LichHoc, được liên kết trực tiếp để quản lý chi tiết về phòng học và thời khóa biểu của từng lớp.

Trong toàn bộ cấu trúc đồ án, bảng DangKyHocPhan được nhóm xác định là bảng trung tâm và đóng vai trò lưu trữ cốt lõi nhất của toàn bộ hệ thống. Bảng này không chứa nhiều thông tin văn bản dài dòng mà chủ yếu lưu trữ các mã số liên kết nhằm giải quyết bài toán quan hệ nhiều nhiều giữa đối tượng sinh viên và đối tượng học phần. Mỗi khi một sinh viên thao tác ghi danh thành công, một dòng dữ liệu mới sẽ lập tức được ghi nhận vào đây. Tiếp theo là bảng BangDiem, nơi lưu trữ toàn bộ các đầu điểm thành phần của sinh viên. Điểm đặc biệt trong cấu trúc bảng BangDiem là nhóm không bắt giáo viên tự tính điểm tổng kết bằng tay, mà thiết kế một cột tự động sinh Generated Column có sẵn công thức nhân hệ số để phần mềm tự động cộng và chia điểm trung bình ngay khi người dùng vừa nhập xong điểm thành phần. Cuối cùng là bảng LogHoatDong, bảng này hoạt động âm thầm dưới hệ thống để ghi chép lại lịch sử mỗi khi có thao tác sửa đổi dữ liệu quan trọng nhằm giúp nhóm dễ dàng truy vết sự cố.

Để đảm bảo toàn bộ tám bảng dữ liệu này hoạt động phối hợp trơn tru và không sinh ra dữ liệu rác trong quá trình sử dụng thực tế, nhóm đã thiết lập một bộ Khóa chính Primary Key và Khóa ngoại Foreign Key cực kỳ chặt chẽ. Khóa chính giúp hệ thống định danh độc nhất cho từng dòng dữ liệu trong mỗi bảng, trong khi khóa ngoại tạo ra các đường liên kết bắt buộc giữa các bảng với nhau nhằm đảm bảo tính toàn vẹn dữ liệu. Mọi thao tác thêm sửa xóa liên kết chéo đều bị hệ thống ràng buộc kiểm tra nghiêm ngặt. Ví dụ, phần mềm Database sẽ không cho phép chèn thêm một lượt đăng ký vào bảng DangKyHocPhan nếu mã sinh viên đó hoàn toàn là mã ảo không tồn tại trong bảng SinhVien. Tương tự, hệ thống cũng sẽ tự động chặn đứng các thao tác lỡ tay xóa một học phần nếu bên trong đó vẫn còn chứa dữ liệu của các sinh

viên đang theo học. Nhờ cấu trúc bảng chuẩn mực và các đường liên kết khóa được ràng buộc kỹ lưỡng ngay từ khâu khởi tạo, cơ sở dữ liệu của nhóm luôn được giữ trong trạng thái nhất quán và an toàn tuyệt đối.

CHƯƠNG 3: CÀI ĐẶT CÁC ĐỐI TƯỢNG CƠ SỞ DỮ LIỆU

3.1. Các hàm tính toán tự động

Trong phần thiết lập đối tượng cho cơ sở dữ liệu, nhóm chúng em đã tiến hành viết mã lệnh tạo ra tổng cộng sáu hàm Functions nhằm mục đích tính toán và tự động hóa các nghiệp vụ xử lý ngầm bên trong hệ thống Database. Hàm đầu tiên mang tên `f_HashPassword` có chức năng cực kỳ quan trọng trong khâu bảo mật thông tin, hàm này sẽ đảm nhiệm việc băm chuỗi mật khẩu ban đầu của người dùng thành một dãy ký tự phức tạp dựa theo chuẩn mã hóa SHA2-256. Tiếp theo là nhóm các hàm chuyên trách về việc tính toán điểm số. Nhóm đã xây dựng hàm `f_TinhDiemTB` để tự động tính điểm trung bình cho từng môn học dựa vào công thức lấy điểm chuyên cần nhân với mười phần trăm cộng điểm giữa kỳ nhân ba mươi phần trăm và cộng điểm cuối kỳ nhân sáu mươi phần trăm. Nhằm hỗ trợ phòng đào tạo theo dõi năng lực sinh viên, nhóm viết thêm hàm `f_TinhGPA` giúp tự động quét toàn bộ bảng điểm, đếm số tín chỉ và tính ra kết quả trung bình tích lũy GPA theo chuẩn thang điểm 4.0. Sau đó, kết quả GPA này sẽ được đưa tiếp vào hàm `f_XepLoaiHocLuc` để tự động dịch từ điểm số sang mức xếp loại học lực bằng chữ như Giỏi, Khá hoặc Trung Bình theo đúng quy chế đào tạo hiện hành của nhà trường. Hai hàm cuối cùng được nhóm thiết kế chuyên biệt để phục vụ trực tiếp cho các luồng kiểm tra dữ liệu khi sinh viên nhấn nút đăng ký môn học. Cụ thể là hàm `f_KiemTraTrungLich` dùng để đối chiếu xem thời khóa biểu của môn học mới có bị cản giờ với các môn mà sinh viên đã lưu trước đó hay không. Cùng với đó là hàm `f_TongTinChiDaDangKy` có nhiệm vụ cộng dồn tổng số tín chỉ hiện tại của sinh viên, qua đó giúp hệ thống phát hiện và báo lỗi ngay nếu sinh viên cố tình đăng ký vượt mức giới hạn cho phép trong một học kỳ.

3.2. Các thủ tục lưu trữ

Tiếp nối phần xây dựng các hàm tính toán, nhóm chúng em đã lập trình một bộ bao gồm mười Stored Procedures nhằm đóng gói toàn bộ các nghiệp vụ phức tạp nhất của hệ thống cơ sở dữ liệu. Trong số đó, thành phần quan trọng nhất và được xem là trái tim của đồ án chính là thủ tục `sp_DangKyHocPhan`. Thủ tục này chịu trách nhiệm xử lý luồng kiểm tra dữ liệu cực kỳ khắt khe trước khi cho phép sinh viên ghi danh vào một lớp học. Cụ thể, khi có một lệnh đăng ký được gửi xuống Database, Stored Procedure này sẽ tự động chạy các bước kiểm tra xem lịch học mới có bị trùng thời gian hay không, đồng thời đối chiếu tổng số tín chỉ hiện tại xem có vượt quá giới hạn tối đa cho phép trong một học kỳ hay không. Nếu phát hiện bất kỳ một điều kiện nào

không thỏa mãn, hệ thống sẽ lập tức báo lỗi và chặn đứng lệnh chen dữ liệu, tuyệt đối từ chối lưu thông tin sai lệch vào bảng đăng ký.

Bên cạnh thủ tục cốt lõi đó, nhóm cũng thiết kế các Stored Procedures thiết yếu khác để giải quyết các chức năng thao tác cơ bản hàng ngày của người dùng. Thủ tục `sp_DangNhap` được xây dựng để xác thực tài khoản một cách an toàn cho cả sinh viên và giáo viên khi bắt đầu truy cập vào hệ thống, đi kèm với đó là thủ tục `sp_DoiMatKhau` giúp người dùng chủ động thay đổi mật khẩu định kỳ. Để phục vụ cho khâu quản lý đào tạo, nhóm đã viết thủ tục `sp_ThemSinhVien` dành riêng cho quyền Admin thao tác khi cần cấp mới tài khoản cho tân sinh viên. Đối với nghiệp vụ điểm số, thủ tục `sp_CapNhatDiem` được tạo ra chỉ cấp quyền cho giáo viên thực hiện việc nhập điểm lần đầu hoặc chỉnh sửa điểm số cho lớp học do mình phụ trách. Ngoài ra, khi sinh viên có nhu cầu rút môn học giữa chừng, hệ thống sẽ gọi thủ tục `sp_HuyDangKyHocPhan` để gỡ bỏ dữ liệu và giải phóng chỗ trống một cách an toàn.

Cuối cùng, nhằm tối ưu hóa tốc độ tải dữ liệu lên giao diện phần mềm ứng dụng, nhóm đã chuyển đổi các câu lệnh truy vấn phức tạp thành các thủ tục chuyên dụng dùng để đọc dữ liệu. Bộ mã này bao gồm thủ tục `sp_XemDanhSachHocPhan` giúp hiển thị toàn bộ các lớp đang mở để sinh viên chọn lựa, thủ tục `sp_XemLichHocSinhVien` dùng để xuất thời khóa biểu cá nhân chuẩn xác, và thủ tục `sp_XemBangDiem` để sinh viên tự theo dõi chi tiết các cột điểm thành phần cùng với điểm tích lũy GPA của mình. Đồng thời, nhóm cũng tạo thêm thủ tục `sp_ThongKeSinhVienTheoHP` nhằm hỗ trợ giáo viên xuất nhanh danh sách toàn bộ sinh viên đang theo học trong một lớp cụ thể. Việc chuyển đổi toàn bộ các nghiệp vụ này thành các Stored Procedures riêng biệt không chỉ giúp mã nguồn hệ thống dễ quản lý hơn mà còn tăng cường độ an toàn bảo mật, giúp ngăn chặn các lỗi phát sinh trong quá trình vận hành thực tế.

3.3. Các cơ chế kích hoạt tự động Triggers

Để hệ thống có thể tự động hóa tối đa các thao tác lặp đi lặp lại và thiết lập hàng rào bảo vệ dữ liệu ở mức cơ sở dữ liệu, nhóm chúng em đã lập trình tổng cộng sáu cơ chế kích hoạt tự động Triggers chạy ngầm. Nhóm Triggers đầu tiên đóng vai trò kiểm soát luồng dữ liệu khi sinh viên bắt đầu ghi danh. Cụ thể, Trigger mang tên `trg_BEFORE_DangKy_KiemTra` sẽ chạy trước mỗi lệnh chen dữ liệu vào bảng đăng ký để kiểm tra nghiêm ngặt xem lớp học đã đầy sĩ số hay chưa và lịch học có bị trùng hay không. Nếu thông tin hợp lệ và sinh viên đăng ký thành công, hệ thống sẽ tiếp tục gọi Trigger mang tên `trg_AFTER_DangKy_CapNhatSiSo`. Cơ chế này sẽ tự động cộng thêm một vào cột sĩ số hiện tại của bảng học phần, giúp con số này luôn nhảy tự động và hiển thị đúng với thực tế mà không cần người quản trị phải tự chạy lệnh cập nhật thủ công. Tương tự như vậy, khi có sinh viên rút môn học, Trigger mang tên

trg_AFTER_HuyDangKy_CapNhatSiSo sẽ lập tức được kích hoạt để tự động trừ đi một khỏi sĩ số hiện tại của lớp học đó.

Bên cạnh việc tự động tính toán, nhóm còn sử dụng Triggers như một lớp rào chắn bảo vệ sự toàn vẹn của dữ liệu vô cùng vững chắc. Nhóm đã viết mã lệnh cho Trigger mang tên trg_BEFORE_XoaHocPhan_BaoVe nhằm mục đích ngăn chặn các sai sót trong quá trình thao tác. Nếu người quản trị hệ thống lỡ tay nhấn nút xóa một học phần đang có sinh viên theo học bên trong, Trigger này sẽ phát hiện và lập tức báo lỗi để chặn đứng lệnh xóa, giúp bảo vệ an toàn tuyệt đối cho thông tin của lớp học. Áp dụng cùng một tư duy thiết kế đó, nhóm tạo thêm Trigger mang tên trg_BEFORE_XoaSinhVien_BaoVe để rào chắn và ngăn chặn hoàn toàn các hành vi xóa tài khoản của những sinh viên đang có dữ liệu đăng ký lưu trữ trong hệ thống.

Cuối cùng, nhằm hỗ trợ tối đa cho công tác kiểm tra và quản lý rủi ro của nhà trường, nhóm đã thiết kế một Trigger chuyên dụng mang tên trg_AFTER_InsertDiem_GhiLog. Cơ chế này hoạt động ngầm bên dưới cơ sở dữ liệu tương tự như một tính năng History dùng để theo dõi lịch sử thao tác của hệ thống. Mỗi khi có giáo viên thực hiện lệnh nhập điểm mới hoặc cập nhật thay đổi điểm số cho sinh viên, Trigger này sẽ tự động sao chép lại thông tin hành động đó và lưu trữ vào một bảng nhật ký hoạt động. Nhờ có cơ chế ghi log chạy ngầm này, nhà trường và nhóm quản trị có thể dễ dàng truy vết lại toàn bộ lịch sử chỉnh sửa điểm thi một cách minh bạch nếu sau này có bất kỳ sự cố hay thắc mắc khiếu nại nào phát sinh từ phía sinh viên.

3.4. Các khung nhìn Views phục vụ báo cáo

Trong quá trình xây dựng hệ thống cơ sở dữ liệu, nhóm chúng em nhận thấy nếu mỗi lần người dùng hoặc phòng đào tạo muốn xem thời khóa biểu hay xuất một bảng điểm chi tiết mà phải viết lại những câu lệnh truy vấn phức tạp dùng phép toán JOIN để nối năm đến sáu bảng dữ liệu lớn lại với nhau thì sẽ làm hệ thống phải xử lý rất nặng nề. Do đó để giải quyết triệt để vấn đề này và tối ưu hóa tốc độ xuất dữ liệu, nhóm đã quyết định thiết lập sẵn năm khung nhìn Views phục vụ riêng cho công tác trích xuất báo cáo nhanh. Các khung nhìn Views này hoạt động tương tự như những bảng ảo đã được hệ thống truy vấn sẵn và định dạng lại dữ liệu chuẩn xác từ nhiều nguồn khác nhau. Nhờ việc lập trình sẵn bộ đối tượng này, người dùng hoặc quản trị viên hệ thống chỉ cần sử dụng một câu lệnh truy vấn SELECT cực kỳ ngắn gọn đổ thẳng vào các Views là đã có thể xuất ngay ra được toàn bộ dữ liệu biểu mẫu báo cáo mình cần mà không tốn nhiều thời gian chờ đợi máy chủ tính toán lại từ đầu.

Cụ thể nhóm đã xây dựng năm khung nhìn Views tương ứng với năm nhu cầu báo cáo thiết yếu nhất của nhà trường. Khung nhìn thứ nhất mang tên vw_BangDiemTongHop được nhóm tạo ra để gom toàn bộ các cột điểm thành phần và

điểm tổng kết thành một bảng điểm đầy đủ cho tất cả sinh viên. Khung nhìn thứ hai là vw_LichHocTongHop có nhiệm vụ hiển thị thời khóa biểu tổng hợp đã được sắp xếp gọn gàng theo từng phòng học và thời gian cụ thể thay vì người dùng phải tự dò tìm kết nối trong nhiều bảng khác nhau. Khung nhìn thứ ba có tên vw_ThongKeHocPhan chuyên dùng để ban quản lý theo dõi và thống kê chi tiết về tình hình sĩ số hiện tại cũng như điểm trung bình của từng lớp học phần đang được mở. Bên cạnh đó nhóm cũng lập trình thêm khung nhìn thứ tư mang tên vw_XepHangSinhVien giúp tự động sắp xếp thứ hạng điểm tích lũy GPA cho toàn bộ sinh viên trong toàn trường để hỗ trợ phòng đào tạo dễ dàng xét duyệt học bổng. Cuối cùng là khung nhìn thứ năm mang tên vw_LogHoatDongChiTiet được thiết kế chuyên biệt để ban quản trị đọc và theo dõi lại toàn bộ nhật ký hoạt động chi tiết của hệ thống nhằm dễ dàng truy vết lịch sử thao tác của người dùng mỗi khi có sự cố mâu thuẫn dữ liệu xảy ra.

CHƯƠNG 4: XỬ LÝ GIAO DỊCH VÀ KIỂM SOÁT ĐỒNG THỜI TRANSACTIONS

4.1. Bảng thiết lập Mức cô lập Isolation Levels

Để đảm bảo hệ thống đăng ký môn học hoạt động chính xác khi có hàng ngàn sinh viên cùng truy cập vào một thời điểm, nhóm chúng em đã nghiên cứu và trình bày chi tiết bảng so sánh các mức cô lập Isolation Levels của cơ sở dữ liệu. Mức cô lập hiệu đơn giản là những quy tắc được hệ thống thiết lập để kiểm soát mức độ mà các giao dịch Transactions có thể can thiệp và làm sai lệch dữ liệu của nhau trong quá trình chạy song song. Trong hệ quản trị cơ sở dữ liệu MySQL mà nhóm sử dụng, có tổng cộng bốn mức cô lập chính được xếp từ thấp đến cao để xử lý các loại lỗi đồng thời khác nhau. Mức thấp nhất trong thang đo này là READ UNCOMMITTED. Ở mức này, hệ thống sẽ cho phép một giao dịch đọc được những dữ liệu đang bị chỉnh sửa dang dở bởi một người khác dù dữ liệu đó chưa hề được lưu chính thức. Điều này cực kỳ nguy hiểm vì nó trực tiếp gây ra lỗi đọc dữ liệu bẩn Dirty Read, làm hiển thị những số liệu ảo không hề tồn tại trong cơ sở dữ liệu.

Để khắc phục tình trạng trên, mức cô lập thứ hai mang tên READ COMMITTED được sinh ra nhằm mục đích ngăn chặn hoàn toàn lỗi Dirty Read. Khi cài đặt cơ sở dữ liệu ở mức này, hệ thống chỉ cho phép người dùng đọc những thông tin đã được xác nhận lưu thành công thông qua lệnh COMMIT. Tuy nhiên, mức cô lập này vẫn còn tồn tại một lỗ hổng nghiêm trọng, đó là nếu đang trong quá trình chạy một báo cáo dài mà có người khác nhảy vào cập nhật dữ liệu, thì kết quả đọc lần một và lần hai sẽ cho ra hai đáp án hoàn toàn khác nhau, đây chính là nguyên nhân trực tiếp gây ra lỗi đọc không lặp lại được Non repeatable Read. Kế tiếp là mức cô lập thứ ba mang tên REPEATABLE READ. Đây chính là mức cô lập mặc định của bộ máy lưu trữ InnoDB mà nhóm chúng em quyết định khẳng định sử dụng làm nền tảng

chuẩn cho toàn bộ hệ thống đồ án của mình. Ở mức cô lập này, mỗi khi một giao dịch bắt đầu chạy thống kê, hệ thống sẽ tự động chụp lại một bức ảnh Snapshot toàn bộ trạng thái dữ liệu tại thời điểm đó. Nhờ cơ chế này, hệ thống có thể ngăn chặn thành công và triệt để cả hai lỗi là Dirty Read và Non repeatable Read.

Mặc dù REPEATABLE READ bảo vệ dữ liệu rất an toàn trước các lệnh chỉnh sửa, nhưng nó vẫn có nguy cơ mắc phải lỗi đọc thấy dòng dữ liệu bóng ma Phantom Read do có người dùng khác thực hiện lệnh chèn thêm dòng dữ liệu mới INSERT vào hệ thống. Để chặn đứng tuyệt đối mọi rủi ro có thể xảy ra, cơ sở dữ liệu cung cấp mức cô lập ở cấp độ cao nhất là SERIALIZABLE. Khi kích hoạt mức này, hệ thống sẽ tạo ra cơ chế khóa phạm vi Range Lock, bắt buộc các giao dịch phải treo lại và xếp hàng chạy lần lượt từng cái một, qua đó ngăn chặn thành công cả ba lỗi Dirty Read, Non repeatable Read và Phantom Read. Sau khi phân tích ưu nhược điểm của cả bốn mức trên, nhóm chúng em nhận thấy việc sử dụng mặc định mức REPEATABLE READ là sự lựa chọn tối ưu nhất để cân bằng giữa hiệu suất hoạt động và độ an toàn cho đồ án. Thay vì đẩy toàn bộ hệ thống lên mức cao nhất làm chậm tốc độ xử lý, nhóm sẽ kết hợp sử dụng linh hoạt các cấp độ cô lập này cùng với các kỹ thuật khóa dòng dữ liệu để khắc phục triệt để năm lỗi giao dịch thực tế sẽ được trình bày chi tiết ở phần tiếp theo.

4.2. Demo và xử lý 5 lỗi Transaction thực tế

Lỗi đầu tiên mà hệ thống của nhóm em phải xử lý là lỗi Lost Update hay còn gọi là mất dữ liệu cập nhật. Giả sử có một lớp học phân hiện đang có ba mươi lượt đăng ký. Khi sinh viên A và sinh viên B cùng lúc nhấn nút đăng ký môn học này, cả hai giao dịch sẽ cùng truy xuất vào cơ sở dữ liệu, cùng đọc được con số ba mươi và tự động tính toán cộng thêm một thành ba mươi một trên bộ nhớ của máy tính cá nhân. Tuy nhiên do hệ thống không có cơ chế khóa, giao dịch của sinh viên B chạy chậm hơn một chút nên đã tiến hành lưu đề kết quả lên phía trên giao dịch của sinh viên A. Hậu quả là tổng số lượng sĩ số cuối cùng ghi nhận trong hệ thống chỉ là ba mươi một thay vì ba mươi hai như thực tế, khiến sinh viên A bị mất đi một lượt đăng ký. Để khắc phục triệt để tình trạng ghi đề này, nhóm em đã áp dụng kỹ thuật Pessimistic Locking bằng cách thêm câu lệnh FOR UPDATE để khóa cứng dòng dữ liệu lại. Đồng thời nhóm yêu cầu cơ sở dữ liệu phải thực hiện phép toán cập nhật trực tiếp và nguyên tử bằng lệnh UPDATE tăng biến sĩ số lên một đơn vị, giúp kết quả luôn được tính đúng thành ba mươi hai.

Lỗi thứ hai được trình bày trong kịch bản báo cáo là lỗi Dirty Read hay còn gọi là đọc dữ liệu bẩn. Tình huống này xảy ra khi điểm thi ban đầu của một sinh viên đang là 7.50. Một giáo viên truy cập vào hệ thống để sửa điểm thành 9.0 nhưng do đang thao tác nên chưa hề nhấn nút lưu hay gọi lệnh COMMIT. Lúc này nếu một sinh viên

khác truy cập vào xem bảng điểm với mức cô lập thấp nhất là READ UNCOMMITTED thì hệ thống sẽ hiển thị cho sinh viên đó thấy con số 9.0, và đây hoàn toàn là dữ liệu rác chưa được xác nhận hợp lệ. Ngay sau đó giáo viên phát hiện mình nhập nhầm nên đã nhấn nút hủy bằng lệnh ROLLBACK khiến điểm số quay ngược trở về mốc 7.50 ban đầu. Điều này đồng nghĩa với việc sinh viên kia vừa đọc được một thông tin ảo không hề tồn tại trong cơ sở dữ liệu. Cách giải quyết của nhóm là nâng cấp cấu hình hệ thống lên mức cô lập SET ISOLATION LEVEL READ COMMITTED để mọi giao dịch chỉ được phép đọc những dữ liệu đã được xác nhận lưu thành công, từ đó loại bỏ hoàn toàn các giao dịch rác.

Lỗi thứ ba nhóm gặp phải là Non repeatable Read tức là lỗi đọc không lặp lại được. Lỗi này thường xuất hiện khi cơ sở dữ liệu đang được cấu hình ở mức READ COMMITTED. Cụ thể là khi quản trị viên đang chạy một thống kê dài, tại lần đọc dữ liệu thứ nhất hệ thống báo điểm của sinh viên là 7.50. Nhưng trong lúc phần mềm còn đang chạy báo cáo chưa xong thì một giáo viên khác nhảy vào cập nhật điểm thành 9.0 và thực hiện lệnh COMMIT thành công. Khi tiến trình báo cáo tiếp tục chạy đến lệnh đọc lần thứ hai thì mức điểm đột nhiên biến thành 9.0. Điều này làm cho cùng một phiên chạy thống kê, cùng một người dùng nhưng lại cho ra hai kết quả dữ liệu khác nhau gây sai lệch toàn bộ thuật toán tính toán. Để giải quyết dứt điểm rủi ro này, nhóm đã sử dụng chuẩn mức cô lập mặc định của bộ máy InnoDB là REPEATABLE READ. Khi đó hệ thống sẽ tự động tạo ra một bản Snapshot chụp ảnh lại toàn bộ trạng thái dữ liệu tại đúng thời điểm bắt đầu chạy báo cáo. Nhờ vậy dù bên ngoài có ai sửa đổi thông tin thì kết quả hiển thị bên trong tiến trình báo cáo vẫn luôn được giữ nguyên vẹn ở mức 7.50.

Lỗi thứ tư là lỗi Phantom Read hay còn gọi là đọc thấy dòng dữ liệu bóng ma. Tình huống này xảy ra khi một giáo vụ đang chạy lệnh đếm số lượng sinh viên đăng ký của một lớp học phần cụ thể, và ở lần đếm đầu tiên hệ thống trả về kết quả là ba mươi người. Đúng lúc này lại có một sinh viên mới thao tác nhấn nút đăng ký thành công làm hệ thống chạy lệnh INSERT chèn thêm một dòng dữ liệu mới vào bảng. Đến cuối kỳ báo cáo, giáo vụ nhấn lệnh đếm lại lần nữa thì con số đột nhiên lòi ra thành ba mươi một người. Sự xuất hiện của một dòng dữ liệu từ hư không này được gọi là bóng ma và nó làm sai lệch hoàn toàn thuật toán thống kê. Để chặn đứng lỗi này, nhóm đã sử dụng mức độ cô lập cao nhất là SERIALIZABLE. Cấp độ này sẽ tạo ra một cơ chế khóa phạm vi Range Lock, có nghĩa là khi quản trị viên đang chạy lệnh đếm số lượng thì mọi hành vi chèn thêm dữ liệu INSERT vào bảng của lớp học phần đó sẽ bị hệ thống treo lại, bắt buộc phải xếp hàng chờ đến khi báo cáo chạy xong mới được thực thi.

Lỗi cuối cùng cũng là lỗi phức tạp nhất nhóm đưa vào báo cáo là lỗi bế tắc khóa hay thuật ngữ chuyên ngành gọi là Deadlock. Lỗi này mô tả tình huống mà hai

giao dịch tiến hành khóa chéo tài nguyên của nhau và mãi mãi chờ đợi đối phương thả khóa. Cụ thể là trong lúc thao tác, sinh viên A đã khóa bảng môn Toán và đang đứng chờ hệ thống cấp quyền để đăng ký môn Lý. Trong khi đó ở một phiên làm việc song song, sinh viên B lại vừa khóa bảng môn Lý và đang chờ hệ thống nhả bảng môn Toán ra để hoàn tất đăng ký. Khi cả hai bên đều rơi vào trạng thái bế tắc, cơ chế phát hiện Deadlock của MySQL sẽ tự động nhảy vào can thiệp. Hệ thống sẽ ép buộc hủy và Rollback một trong hai giao dịch, đồng thời trả về mã lỗi 1213 mang thông báo Deadlock found, khi đó giao dịch của người còn lại mới được giải phóng để tiếp tục chạy. Để khắc phục triệt để tình trạng bế tắc này, nhóm em đã thiết lập quy tắc trong mã nguồn Backend là toàn bộ các câu truy vấn phải luôn thực hiện khóa bảng theo một thứ tự cố định thống nhất. Ngoài ra nhóm cũng lập trình thêm một khối lệnh để phần mềm có thể tự động Retry thử lại giao dịch ngay lập tức khi phát hiện hệ thống văng ra lỗi 1213.

CHƯƠNG 5: HƯỚNG DẪN TRIỂN KHAI VÀ KỊCH BẢN DEMO

5.1. Cấu trúc Source Code với 6 Script Khởi tạo

Để quá trình quản lý mã nguồn trở nên khoa học và thuận tiện cho việc bàn giao hệ thống sau này, thay vì gom tất cả mã lệnh SQL vào một tệp duy nhất rất dài và khó kiểm soát, nhóm chúng em đã quyết định phân rã toàn bộ cấu trúc cơ sở dữ liệu thành sáu file script riêng biệt. Các file này được đặt tên và đánh số thứ tự rõ ràng từ 01 đến 06, yêu cầu người triển khai phải chạy lần lượt từng file một trên phần mềm MySQL Workbench để đảm bảo hệ thống khởi tạo đúng luồng nghiệp vụ. Việc chia nhỏ mã nguồn thành từng cụm chức năng như thế này giúp nhóm dễ dàng phân công công việc lập trình cho từng thành viên và nhanh chóng khoanh vùng được lỗi nếu có sự cố phát sinh trong lúc thuyết trình bảo vệ đồ án.

File mã nguồn đầu tiên mang tên 01_CreateDatabase.sql đóng vai trò đặt nền móng cho toàn bộ hệ thống. Khi thực thi file này, máy chủ sẽ tự động tạo ra một cơ sở dữ liệu mới tinh bao gồm cấu trúc của tám bảng chính là bảng SinhVien, GiaoVien, MonHoc, HocPhan, LichHoc, DangKyHocPhan, BangDiem và LogHoatDong. Tại file này nhóm em cũng đã lập trình cấu hình sẵn toàn bộ các ràng buộc khóa chính Primary Key và khóa ngoại Foreign Key cho các bảng, tạo ra một bộ khung liên kết chặt chẽ để hệ thống tự động ngăn chặn tình trạng xuất hiện dữ liệu rác. Kế tiếp là file 02_SampleData.sql chuyên dùng để nạp dữ liệu giả lập phục vụ quá trình kiểm thử. Nhóm đã chuẩn bị sẵn các câu lệnh INSERT để tự động nạp vào cơ sở dữ liệu mười lăm tài khoản sinh viên, năm giáo viên, mười môn học cùng hàng loạt học phần mẫu. Điểm đặc biệt của file nạp dữ liệu này là nhóm có chèn thêm lệnh tắt kiểm tra khóa ngoại SET FOREIGN KEY CHECKS bằng không kết hợp cùng lệnh làm sạch bảng TRUNCATE ở ngay những dòng đầu tiên. Kỹ thuật này giúp nhóm có thể tự động xóa

sạch toàn bộ dữ liệu cũ và reset hệ thống về trạng thái ban đầu chỉ trong nháy mắt nếu lỡ thao tác sai trong lúc demo cho giảng viên xem.

Sau khi đã có bộ khung bảng và dữ liệu mẫu, hệ thống cần được nạp các khối lệnh xử lý logic. File thứ ba là 03_Functions.sql chứa tổng cộng sáu hàm Functions chuyên dùng để tính toán tự động. Điển hình trong đó là hàm f_TinhGPA tự động quét các điểm thành phần để tính ra điểm trung bình tích lũy, hay hàm f_XepLoaiHocLuc hỗ trợ tự động dịch từ điểm số sang chữ như Giỏi Khá Trung Bình chuẩn theo quy chế nhà trường. Tiếp nối ngay sau là file 04_StoredProcedures.sql được nhóm xem như là trái tim vận hành của hệ thống. File này tập hợp mười thủ tục lưu trữ Stored Procedures xử lý toàn bộ các nghiệp vụ khó nhất, trong đó quan trọng nhất là thủ tục sp_DangKyHocPhan. Khi sinh viên bấm lệnh đăng ký, thủ tục này sẽ tự động rà soát xem lớp học mới có bị trùng lịch hay không và kiểm tra xem tổng tín chỉ có bị vượt quá hạn mức cho phép hay không. Nếu phát hiện bất kỳ vi phạm nào, phần mềm sẽ lập tức báo lỗi và tuyệt đối từ chối lệnh chèn dữ liệu vào bảng.

Để hoàn thiện khả năng tự động hóa, nhóm em thiết lập file thứ năm là 05_Triggers.sql chứa các cơ chế kích hoạt chạy ngầm. Nhờ các mã lệnh Triggers này, mỗi khi có một dòng đăng ký thành công được thêm vào bảng, hệ thống sẽ tự động cộng thêm một đơn vị vào cột số hiện tại của bảng học phần tương ứng. Đồng thời file này cũng cài đặt tính năng History, tự động ghi chép lại mọi hành động sửa điểm thi hay đổi mật khẩu vào bảng lưu trữ nhật ký hoạt động. Cuối cùng là file 06_Views.sql chứa các khung nhìn Views được biên dịch sẵn nhằm phục vụ riêng cho khâu xuất báo cáo. Thay vì mỗi lần người dùng muốn xem thời khóa biểu hệ thống lại phải chạy lệnh JOIN để nối năm hoặc sáu bảng lại với nhau cực kỳ nặng nề và làm chậm máy chủ, nhóm em đã tạo sẵn các View như v_ThoiKhoaBieu_SinhVien. Từ đây về sau, ứng dụng quản lý chỉ cần thực thi một câu lệnh truy vấn SELECT đơn giản trở thẳng vào các View này là đã có thể xuất ra ngay giao diện thời khóa biểu cực kỳ nhanh chóng và chính xác.

5.2. Hướng dẫn chạy và Tài khoản Test

Để triển khai và chạy thử nghiệm hệ thống cơ sở dữ liệu này trên máy tính cá nhân, người dùng cần cài đặt sẵn phần mềm MySQL Server phiên bản 8.0 trở lên cùng với công cụ quản trị trực quan MySQL Workbench. Sau khi cài đặt hoàn tất, thao tác chạy mã nguồn được thực hiện vô cùng đơn giản thông qua các thanh công cụ có sẵn của phần mềm. Cụ thể, người dùng chỉ cần nhấn vào thanh menu chọn File, tiếp tục chọn lệnh Open SQL Script để duyệt và mở lần lượt từng file mã nguồn mà nhóm em đã đóng gói sẵn. Khi file mã lệnh hiển thị trên màn hình, người dùng nhấn chọn nút Execute để máy chủ bắt đầu thực thi các dòng lệnh. Một điểm quan trọng bậc nhất cần lưu ý là toàn bộ sáu file script này bắt buộc phải được người dùng chạy theo đúng số

thứ tự từ file 01 đến file 06 để hệ thống có thể khởi tạo bộ khung cấu trúc bảng trước khi tiến hành nạp dữ liệu mẫu và các hàm xử lý logic.

Trong quá trình thao tác demo trực tiếp cho giảng viên chấm điểm, nhóm em đã lường trước các tình huống có thể thao tác sai bước hoặc vô tình làm rác dữ liệu của hệ thống. Để giải quyết vấn đề này một cách nhanh chóng và chuyên nghiệp, nhóm em đã thiết lập một cơ chế làm sạch và nạp lại dữ liệu cực kỳ tiện lợi ngay bên trong file lệnh chứa dữ liệu mẫu. Tại những dòng lệnh đầu tiên của file này, nhóm đã chủ động chèn thêm mã lệnh SET FOREIGN KEY CHECKS bằng không để tạm thời ngắt hệ thống kiểm tra khóa ngoại, kết hợp cùng lệnh TRUNCATE chuyên dụng để dọn dẹp và xóa sạch toàn bộ dữ liệu đang tồn tại trong các bảng. Nhờ vào tư duy thiết kế tinh tế này, nếu đồ án có lỡ gặp bất kỳ lỗi mâu thuẫn dữ liệu nào khi đang thuyết trình bảo vệ, nhóm em chỉ việc mở file nạp dữ liệu lên và bấm nút chạy lại một lần duy nhất là toàn bộ hệ thống cơ sở dữ liệu sẽ lập tức được reset quay trở về trạng thái sạch sẽ như lúc ban đầu.

Để phục vụ trực tiếp cho việc kiểm thử các luồng nghiệp vụ trên phần mềm, nhóm em cũng đã chuẩn bị sẵn một bộ dữ liệu tài khoản mẫu bao gồm hai nhóm người dùng nòng cốt là sinh viên và giáo viên. Đối với nhóm người dùng sinh viên, hệ thống đã nạp sẵn mười lăm tài khoản được đánh mã định danh lần lượt từ SV001 đến SV015 đi kèm với mật khẩu mặc định cực kỳ dễ nhớ là sv123456. Tương tự như vậy, đối với nhóm người dùng đóng vai trò giảng dạy, nhóm em đã thiết lập năm tài khoản được đánh mã từ GV001 đến GV005 với mật khẩu đăng nhập mặc định là gv123456. Điều đặc biệt để lấy điểm cộng cho đồ án là mặc dù mật khẩu cung cấp để test trên giao diện rất đơn giản, nhưng ngay khi được nạp sâu vào trong cơ sở dữ liệu, tất cả các chuỗi mật khẩu này đều đã được hệ thống mã hóa hoàn toàn thành các chuỗi ký tự vô cùng phức tạp dựa theo chuẩn bảo mật SHA2 256. Thiết lập kỹ thuật này giúp đồ án của nhóm thể hiện tính ứng dụng thực tế sát với môi trường doanh nghiệp và đảm bảo được độ an toàn cao nhất trong khâu quản lý thông tin nhạy cảm của người dùng.

5.3. Kịch bản Demo thực tế trình chiếu

Trong phần cuối cùng của buổi thuyết trình đồ án, nhóm chúng em đã chuẩn bị một kịch bản demo thực tế trực tiếp trên phần mềm MySQL Workbench để trình chiếu cho giảng viên chấm điểm xem toàn bộ luồng hoạt động của hệ thống cơ sở dữ liệu. Đầu tiên nhóm sẽ mở giao diện phần mềm và trình bày nhanh qua cấu trúc sáu file mã nguồn đã được chia tách rõ ràng để giảng viên thấy được cách tổ chức code khoa học của nhóm. Tiếp theo nhóm tiến hành mở một tab truy vấn mới để bắt đầu chạy thử nghiệm các tính năng tự động hóa cốt lõi nhất. Khởi đầu kịch bản nhóm sẽ demo việc gọi hàm tính điểm trung bình tích lũy GPA cho một tài khoản cụ thể là sinh viên mang mã số SV001. Khi dòng lệnh truy vấn được quét và thực thi, hệ thống sẽ tự động tổng

hợp toàn bộ bảng điểm của sinh viên này, tính toán ra con số chính xác và xuất luôn ra kết quả chữ kèm theo là xếp loại Giỏi hoàn toàn tự động.

Sau khi demo thành công phần hàm tính toán số liệu, nhóm tiếp tục chuyển sang trình diễn tính năng kích hoạt chạy ngầm Triggers. Nhóm sẽ mở bảng học phần lên cho giảng viên xem trước thông tin của lớp học mang mã HP002 hiện tại đang có mức sĩ số hiển thị là bốn người. Ngay sau đó nhóm sẽ đóng vai một sinh viên mới mang mã số SV014 và tiến hành gõ lệnh chèn dữ liệu đăng ký thêm vào đúng lớp học phần này. Khi lệnh đăng ký vừa báo thành công, nhóm sẽ lập tức truy vấn lại bảng học phần để giảng viên thấy rõ con số sĩ số hiển thị tại đã tự động nhảy lên thành năm người nhờ vào cơ chế Trigger chạy ngầm bên dưới mà không cần quản trị viên phải gõ thêm bất kỳ câu lệnh cập nhật thủ công nào. Các cơ chế tự động ghi log lịch sử khi xóa sửa điểm cũng sẽ được nhóm thao tác thêm sửa trực tiếp để chứng minh hệ thống hoạt động đúng như thiết kế.

Phần trình diễn cuối cùng và cũng là bước trọng tâm để nhóm lấy điểm tuyệt đối chính là phần thao tác thực tế nắm tình huống lỗi giao dịch Transaction. Nhóm đã chuẩn bị sẵn các file truy vấn chia thành nhiều tab chạy song song để giả lập tình huống có nhiều người dùng cùng lúc truy cập vào hệ thống. Lần lượt nhóm sẽ chạy các mã lệnh để tái hiện lại cảnh hai sinh viên cùng giành nhau một chỗ trống cuối cùng gây ra lỗi mất dữ liệu cập nhật Lost Update, hoặc tình huống sinh viên bị hiển thị mức điểm ảo do dính lỗi đọc dữ liệu bẩn Dirty Read khi giáo viên sửa điểm nhưng chưa nhấn lệnh lưu. Ở mỗi một tình huống demo lỗi sinh ra, nhóm đều sẽ trực tiếp chạy các đoạn code khắc phục bằng cách thiết lập lại các mức cô lập Isolation Levels kết hợp cùng các kỹ thuật khóa dòng dữ liệu chuyên sâu để chứng minh cho giảng viên thấy toàn bộ rủi ro về tranh chấp đồng thời đã được hệ thống rào chắn và giải quyết triệt để.